

Michael Greene - Mo/Wed 4-5:15PM

Question 1:

Steps:

- a.

```
return sum(
    state[i] != self.goal[i]
    for i in range(len(state))
)
```
- b. This lower-bounds the number of remaining actions because a DOGCAT action can change only one letter position at a time. Therefore, if three positions currently differ from the goal, at least three letter replacements are necessary.
- c. Dictionary restrictions may force additional intermediate changes, but they can never allow more than one mismatched position to be corrected by a single action. Therefore, this heuristic cannot overestimate the true remaining cost.

Scrabble:

- a.

```
scrabble_heuristic(state, goal):
    total = 0
    for each letter position i:
        if state[i] != goal[i]:
            total += Scrabble value of goal[i]
    return total
```
- b. This is a lower bound because every incorrect position must eventually have its correct goal letter inserted into that position. The cost of an action is determined by the inserted letter, so inserting each of those goal letters has an unavoidable Scrabble cost.
- c. A valid path may require extra intermediate letters, but those changes only add cost. Therefore, the heuristic cannot exceed the actual minimum remaining Scrabble cost.

Frequency:

- a.

```
frequency_heuristic(state, goal):
    mismatches = 0
    for each letter position i:
        if state[i] != goal[i]:
            mismatches += 1
    return mismatches
```
- b. It lower-bounds the minimum number of remaining actions, since every incorrect position must be changed at least one time.

- c. Every frequency action costs at least 1 because its cost is $1 + \text{rarity}(\text{word})$ and rarity is nonnegative. Therefore, if there are x mismatched positions, the true remaining cost must be at least x

Question 2:

Word Pair	Cost Measure	UCS Cost	UCS Expanded	A* Cost	A* Expanded
WHY -> ASK	Steps	5	773	5	16
WHY -> ASK	Scrabble	9	765	9	13
WHY -> ASK	Frequency	38.8554	808	38.8554	755
BEAR -> DUCK	Steps	4	691	4	6
BEAR -> DUCK	Scrabble	11	2981	11	6
BEAR -> DUCK	Frequency	38.2402	1084	38.2402	720

- Yes. For WHY -> ASK: for all three methods, both returned 5, 9, and 38.8554 respectively. For BEAR -> DUCK: for all three methods, both returned 4, 11, and 38.2402 respectively.
- For WHY -> ASK: Scrabble gave the largest reduction = 98.30%. For BEAR -> DUCK: Scrabble also gave the largest reduction = 99.80%.
- For both pairs, Frequency was the least informative.
- The heuristic for Steps calculates an estimate of the number of unavoidable actions. If three letters are wrong, then at least three actions are required, so the estimate is pretty close to the remaining solution cost. The Scrabble heuristic is even better because it takes into account not only how many positions must change, but also the different unavoidable costs of the goal letters. The Frequency heuristic only counts wrong positions even though an actual frequency action can cost much more than 1. Therefore, the heuristic can be much smaller than the true remaining cost.

Question 3:

Steps: Yes, the optimal cost is symmetric because every action costs 1. Reversing a valid path gives a path with the same number of steps.

Scrabble: No, the cost depends on the letter being inserted, so reversing an action can change its cost. To show this, if we have dog $\rightarrow$ cog which inserts c with a cost of 3, then we can see that it is cheaper for cog $\rightarrow$ dog which inserts d and costs 2.

Frequency: No, the cost depends on the resulting word, so reversing an action changes the rarity of a different word. For example, dog $\rightarrow$ cog costs $1 + \text{rarity}(\text{cog})$, while cog $\rightarrow$ dog costs $1 + \text{rarity}(\text{dog})$ which are not necessarily equal.

Question 4:

1.

- a. GPT-5.6 Sol
- b. Exact Prompt: "In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.
By 'more informative,' we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost."
- c. Proposed heuristic: For each goal position, find the smallest rarity of any legal dictionary word of the correct length that already has the required goal letter in that position. For every mismatched position, add 1 plus that minimum rarity.

2.

Precompute:

for each position i :

$\text{minRarity}[i] =$

minimum $\text{rarity}(w)$ among all legal words w

of the same length as the goal

where $w[i] == \text{goal}[i]$

$\text{rarityHeuristic}(\text{state})$:

$h = 0$

for each position i :

if $\text{state}[i] \neq \text{goal}[i]$:

$h += 1 + \text{minRarity}[i]$

return h

3. The heuristic is admissible. Why this is so is because every incorrect position letter that gets changed into its goal letter must result in a legal word from the dictionary. That means the rarity of that word must be at least $\text{minRarity}[i]$. So an action that corrects an incorrect letter will cost at the very least: $1 + \text{minRarity}[i]$. The heuristic then sums all these unavoidable lower bounds for all incorrect letter positions. Any additional changes only increase the true path cost, meaning the heuristic can't overestimate the optimal remaining cost. Likewise, it also dominates h_0 because all rarity values are nonnegative so: $1 + \text{minRarity}[i] \geq 1$ for every incorrect position. In the provided dictionary, the values of rarity are positive, so this heuristic is strictly larger than h_0 for every non-goal state.
4. No repair is necessary because the heuristic is admissible and satisfies the requirement that it should be at least as informative as h_0 .
- 5.

Problem	Heuristic	Solution Cost	Nodes Expanded
WHY->ASK	h_0	38.8554	755
WHY->ASK	Rarity-aware	38.8554	510
BEAR -> DUCK	h_0	38.2402	720
BEAR -> DUCK	Rarity-aware	38.2402	319

- Both heuristics give the same optimal solution costs but the rarity-aware heuristic expanded far less nodes in both cases. Therefore, this evidence supports using the rarity-aware heuristic instead of h_0 for the frequency objective because it preserves optimality while providing A* with a stronger estimate of the remaining cost.